

Appendix: Code Summary

Appendix: Code Summary

Project Structure

legged_gym

`__init__.py`

envs/base

`base_config.py`

envs/pointfoot

`point_foot.py`

`pointfoot_rough_config.py`

class `PointFootRoughCfg(BaseConfig)`:

class `PointFootRoughCfgPPO(BaseConfig)`

scripts

`export_policy_as_onnx.py`

`play.py`

`train.py`

tests

`test_env.py`

utils

`helpers.py`

class `PolicyExporterLSTM(torch.nn.Module)`:

`logger.py`

`math.py`

`task_registry.py`

`terrain.py`

pointfootMujoco

`r1_controller.py`

1. Initialization (`__init__` method):

2. Configuration Loading (`load_config` method):

3. Main Control Loop (`run` method):

4. Modes of Operation

5. Observations and Actions

6. Callbacks

`simulator.py`

pointfoot-sdk-lowlevel

policy/PF_TRON1A

`policy/policy.onnx`

`params.yaml`

robot_description/PF_TRON1A

robot-joystick

resources

MISC

`install.sh`

`setup.py`

Some parts directly use code comments in TAs' code.

In summary, this template provides a framework for training and visualizing point-foot legged robots using PPO.

It also provides sim-to-sim transfer capabilities between Isaac Gym and Mujoco.

Project Structure

```
|— LICENSE
|— README.md
|— install.sh
|— legged_gym
|   |— __init__.py
|   |— envs
|   |   |— __init__.py
|   |   |— base
|   |   |   |— base_config.py
|   |   |   |— pointfoot
|   |   |       |— point_foot.py
|   |   |       |— pointfoot_rough_config.py
|   |— scripts
|   |   |— export_policy_as_onnx.py
|   |   |— play.py
|   |   |— train.py
|   |— tests
|   |   |— test_env.py
|   |— utils
|   |   |— __init__.py
|   |   |— helpers.py
|   |   |— logger.py
|   |   |— math.py
|   |   |— task_registry.py
|   |   |— terrain.py
|— licenses
|   |— ...
|— pointfootMujoco
|   |— README.md
|   |— pointfoot-sdk-lowlevel # Low-level Controller
|   |   |— ...
|   |— policy
|   |   |— PF_TRON1A
|   |   |   |— params.yaml
|   |   |   |— policy
|   |   |       |— policy.onnx
|   |— rl_controller.py
|   |— robot-joystick
|   |— robot_description
|   |   |— PF_TRON1A
|   |   |   |— meshes # external resources for the robot
|   |   |   |   |— abad_L_Link.STL
|   |   |   |   |— ...
|   |   |   |— xml
|   |   |       |— robot.xml # urdf file
|   |— simulator.py
|— resources # urdf & mesh for different models
|   |— ...
|— setup.py
```

legged_gym

`__init__.py`

Get `ROBOT_TYPE` from command line, if it is "PF", then register task with `PointFoot`, `PointFootRoughCfg()`, `PointFootRoughCfgPPO()`.

envs/base

`base_config.py`

This file contains a class `BaseConfig` that used to initialize all its member classes **recursively**.

- Constructor:
Initialize all member classes by calling `self.init_member_classes(self)`
- static method `init_member_classes(obj)`:
Recursively initialize all member classes of the given object.
 - iterate through all the attributes of the object.
 - retrieve the attribute object using `getattr()`
 - If the attribute is an instance, **instantiate** it and sets the attribute to the instance instead of the type. Then call `init_member_classes()` on the instance (recursively).

envs/pointfoot

`point_foot.py`

Build a environment and agent of point foot robots.

class `PointFoot`: Includes following method.

- method `__init__(self, cfg, sim_params, physics_engine, sim_device, headless)`
 - Parses the provided config file using `_parse_cfg()`
 - Creates, simulation, terrain and environments using `create_sim()`
 - Initializes pytorch buffers used during training using `_init_buffers()`
 - Calls `_prepare_reward_function()`
 - Args:
 - `cfg` (Dict): Environment config file
 - `sim_params` (`gymapi.SimParams`): simulation parameters
 - `physics_engine` (`gymapi.SimType`): `gymapi.SIM_PHYSX` (must be PhysX)
 - `device_type` (string): 'cuda' or 'cpu'
 - `device_id` (int): 0, 1, ...
 - `headless` (bool): Run without rendering if True
- method `get_observations(self)`
return `self.proprioceptive_obs_buf`
- method `get_privileged_observations(self)`
return `self.privileged_obs_buf`

- method `reset(self)`
Reset all robots using `self.reset_idx(torch.arange(self.num_envs, device=self.device))`. And return `obs`, `privileged_obs`
- method `render(self, sync_frame_time=True)`
Render the viewer.
- method `step(self, actions)`
Apply actions and simulate.
 - First clip actions using `torch.clip()`
 - Render the viewer using `self.render()`
 - Simulate using `self.post_physics_step()`
 - return clipped obs, clipped states (None), rewards, dones and infos
 - return `self.proprioceptive_obs_buf`, `self.privileged_obs_buf`, `self.rew_buf`, `self.reset_buf`, `self.extras`
- method `post_physics_step(self)`
Simulation in physics.
- method `_check_if_include_feet_height_rewards(self)`
Check whether `feet_height` in the reward scale directory, and return a Boolean.
- method `check_termination(self)`
Check if environments need to be reset by `self.termination_contact_indices` in `self.contact_forces` and whether time out.
- method `reset_idx(self, env_ids)`
Reset some environments.
 - First reset robot states using `self._reset_dofs(env_ids)`, `self._reset_root_states(env_ids)`, `self._resample(env_ids)`
 - Then fill extras, log additional curriculum info, and send timeout info to the algorithm.
- method `_reset_buffers(self, env_ids)`
Reset buffers.
- method `compute_reward(self)`
Compute rewards. Calls each reward function which had a non-zero scale (processed in `self._prepare_reward_function()`), adds each terms to the episode sums and to the total reward.
- method `compute_observations(self)`
Computes observations. Calls `self.compute_proprioceptive_observations()`, `self.compute_privileged_observations()`, `self._add_noise_to_obs()`.
- method `_add_noise_to_obs(self)`
Add noise if `self.add_noise` is true.
- method `compute_privileged_observations(self)`
Calls `self._compose_privileged_obs_buf_no_height_measure()` and add perceptive inputs if not blind using `self._add_height_measure_to_buf(self.privileged_obs_buf)`.

- method `_compose_privileged_obs_buf_no_height_measure(self)`
Composes privileged observation buffer including:
`self.base_ang_vel * self.obs_scales.ang_vel`
`self.projected_gravity`
`(self.dof_pos - self.default_dof_pos) * self.obs_scales.dof_pos`
`self.dof_vel * self.obs_scales.dof_vel`
`self.actions`
`self.commands[:, :3] * self.commands_scale`
- method `compute_proprioceptive_observations(self)`
Calls `self._compose_proprioceptive_obs_buf_no_height_measure()` and add perceptive inputs if not blind using
`self._add_height_measure_to_buf(self.proprioceptive_obs_buf)`.
- method `_add_height_measure_to_buf(self, buf)`
Heights is `self.root_states[:, 2].unsqueeze(1) - 0.5 - self.measured_heights` clipped in `[-1, 1]`.
- method `_compose_proprioceptive_obs_buf_no_height_measure(self)`
Composes proprioceptive observation buffer including:
`self.base_ang_vel * self.obs_scales.ang_vel`
`self.projected_gravity`
`(self.dof_pos - self.default_dof_pos) * self.obs_scales.dof_pos`
`self.dof_vel * self.obs_scales.dof_vel`
`self.actions`
`self.commands[:, :3] * self.commands_scale`
- method `create_sim(self)`
Creates simulation, terrain and environments.
- method `set_camera(self, position, lookat)`
Set camera position and direction.
- method `_process_rigid_shape_props(self, props, env_id)`
 - Description
Callback allowing to store/change/randomize the rigid shape properties of each environment.
Called During environment creation.
Base behavior: randomizes the friction of each environment.
 - Args:
`props (List[gymapi.RigidShapeProperties])`: Properties of each shape of the asset
`env_id (int)`: Environment id
 - Returns:
`[List[gymapi.RigidShapeProperties]]`: Modified rigid shape properties
- method `_process_dof_props(self, props, env_id)`
 - Description
Callback allowing to store/change/randomize the DOF properties of each environment.
Called During environment creation.
Base behavior: stores position, velocity and torques limits defined in the URDF.

- Args:
 - `props (numpy.array)`: Properties of each DOF of the asset
 - `env_id (int)`: Environment id
- Returns:
 - `[numpy.array]`: Modified DOF properties
- method `_process_rigid_body_props(self, props, env_id)`

Randomize base mass.
- method `_post_physics_step_callback(self)`

Callback called before computing terminations, rewards, and observations.

Default behavior: Compute ang vel command based on target and heading, compute measured terrain heights and randomly push robots.
- method `_resample(self, env_ids)`

Calls `_resample_commands(self, env_ids)`.
- method `_resample_commands(self, env_ids)`

Randomly select commands of some environments.

 - Args:
 - `env_ids (List[int])`: Environments ids for which new commands are needed
- method `_compute_torques(self, actions)`
 - Description:

Compute torques from actions.

Actions can be interpreted as position or velocity targets given to a PD controller, or directly as scaled torques.

[NOTE]: torques must have the same dimension as the number of DOFs, even if some DOFs are not actuated.

Args:

`actions (torch.Tensor)`: Actions

Returns:

`[torch.Tensor]`: Torques sent to the simulation
- method `_reset_dofs(self, env_ids)`
 - Description:

Resets DOF position and velocities of selected environments

Positions are randomly selected within `0.5:1.5 x` default positions.

Velocities are set to zero.
 - Args:
 - `env_ids (List[int])`: Environment ids
- method `_reset_root_states(self, env_ids)`
 - Description:

Resets ROOT states position and velocities of selected environments

Sets base position based on the curriculum

Selects randomized base velocities within `-0.5:0.5 [m/s, rad/s]`
 - Args:
 - `env_ids (List[int])`: Environment ids

- method `_push_robots(self)`

Randomly pushes the robots.

- method `_update_terrain_curriculum(self, env_ids)`

Implements the game-inspired curriculum(adjusts the complexity of the terrain according to the robot's performance).

- Args:

`env_ids (List[int])`: ids of environments being reset

- method `update_command_curriculum(self, env_ids)`

Implements a curriculum of increasing commands(increases the range of commands if the tracking reward is above 80%).

- Args:

`env_ids (List[int])`: ids of environments being reset

- method `_get_noise_scale_vec(self)`

- Description:

Sets a vector used to scale the noise added to the observations.

[NOTE]: Must be adapted when changing the observations structure

- Args:

`cfg (Dict)`: Environment config file

- Returns:

`[torch.Tensor]`: Vector of scales used to multiply a uniform distribution in [-1, 1]

- method `_init_buffers(self)`

Initialize torch tensors which will contain simulation states and processed quantities.

- method `_prepare_reward_function(self)`

Prepares a list of reward functions, which will be called to compute the total reward.

Looks for `self._reward_<REWARD_NAME>`, where `<REWARD_NAME>` are names of all non zero reward scales in the cfg.

- First, remove zero scales and multiply non-zero ones by dt.

- Prepare list of functions.

- Initial the reward episode sums.

- method `_create_ground_plane(self)`

Adds a ground plane to the simulation, sets friction and restitution based on the cfg.

- method `_create_heightfield(self)`

Adds a heightfield terrain to the simulation, sets parameters based on the cfg.

- method `_create_trimesh(self)`

Adds a triangle mesh terrain to the simulation, sets parameters based on the cfg.

- method `_create_envs(self)`

Creates environments:

1.loads the robot URDF/MJCF asset,

2.For each environment

2.1 creates the environment,

2.2 calls DOF and Rigid shape properties callbacks,

2.3 create actor with these properties and add them to the env

3.Store indices of different bodies of the robot

- method `_get_env_origins(self)`

Sets environment origins. On rough terrain the origins are defined by the terrain platforms. Otherwise create a grid.

- method `_parse_cfg(self)`

Parse the config file.

- method `_draw_debug_vis(self)`

Draws visualizations for debugging (slows down simulation a lot).

Default behavior: draws height measurement points

- method `_init_height_points(self)`

Returns points at which the height measurements are sampled (in base frame).

- Returns:

`[torch.Tensor]`: Tensor of shape `(num_envs, self.num_height_points, 3)`

- method `_get_heights(self, env_ids=None)`

- Description:

Samples heights of the terrain at required points around each robot.

The points are offset by the base's position and rotated by the base's yaw

- Args:

`env_ids (List[int], optional)`: Subset of environments for which to return the heights. Defaults to None.

- Returns:

`heights.view(self.num_envs, -1) * self.terrain.cfg.vertical_scale`

- method `_get_terrain_heights_from_points(self, points)`

return `heights`

- method `_compute_feet_states(self)`

Compute the states of feet.

- method `_reward_ang_vel_xy(self)`

Penalize xy axes base angular velocity

- method `_reward_base_height(self)`

Penalize base height away from target

- method `_reward_torques(self)`

Penalize torques

- method `_reward_dof_acc(self)`

Penalize dof accelerations

- method `_reward_action_rate(self)`

Penalize changes in actions

- method `_reward_collision(self)`

Penalize collisions on selected bodies

- method `_reward_torque_limits(self)`

penalize torques too close to the limit

- method `_reward_feet_air_time(self)`
Reward steps between proper duration
- method `_reward_feet_distance(self)`
Reward feet distance
- method `_reward_survival(self)`
Reward survival

`pointfoot_rough_config.py`

This file contains two main configuration classes:

- `PointFootRoughCfg`: Environment parameters
- `PointFootRoughCfgPPO`: Training parameters

class `PointFootRoughCfg(BaseConfig)`:

- inner class `env`:
Mainly contains the number of robots, actions, observations, etc.
- inner class `terrain`:
Params for generating terrain, including size, numbers, proportions, etc.
- inner class `commands`:
Params for the commands, especially the range (boundries of linear and angular velocities, and heading angle) of the commands.
- inner class `init_state`:
Init state of the robot, including the position, orientation, velocity of the base, and the joint positions.
- inner class `control`:
Set control type to "position". And set the stiffness and damping of the PD drive.
- inner class `asset`:
Set the asset path of the robot model based on the `ROBOT_TYPE`. And set other params related to the robot's physical and operational attributes.
- inner class `domain_rand`:
Range for some randomization parameters, including the friction, mass, push, etc.
- **inner class** `rewards`:
Scale coefficients for different rewards.
- inner class `normalization`:
Params for normalization.
- inner class `noise`:
Params for noise.
- inner class `viewer`:
Set the position of viewer.
- inner class `sim`:
Set params for simulation such as time interval `dt` and gravity etc.
Besides that an inner-inner class `physx` contains the params for PhysX engine.

`class PointFootRoughCfgPPO(BaseConfig)`

This class uses `OnPolicyRunner`.

- inner class `policy`:
Set initial noise standard deviation to 1.0, activation function to "ELU" (Exponential Linear Unit).
The dimensionality of **actor** network is $512 \times 256 \times 128$, and for **critic** is also $512 \times 256 \times 128$.
- inner class `algorithm`:
Training params, such as clip value loss, entropy coefficient, etc.
- inner class `runner`:
Specify the policy class as "Actor-Critic" and algorithm class as "PPO". Then set some params for logging and loading.

scripts

`export_policy_as_onnx.py`

- method `export_policy_as_onnx(args)`:
First get environment and training configurations from the `task_registry`. Then set the path and instantiate a actor-critic model. Finally, export the model as an ONNX file by calling `torch.onnx.export`.

`play.py`

This file constains a function `play(args)` and mainly achieves the following functions:

1. Override some parameters for testing. Then create a task environment and a algorithm runner with reduced number of robots and terrains.
2. If set `EXPORT_POLICY` to True, export the policy as a TorchScript module (`export_policy_as_jit(...)`) an ONNX model (`export_policy_as_onnx(...)`).
3. Set some parameters for the logger, start palaying the policy, then log and plot the states.

`train.py`

- method `train(args)`:
 - Create a task **environment** and a **training algorithm runner** using parsed arguments.
 - Train and save the model.

tests

`test_env.py`

Set up a test environment with reduced number of parallel environments. Then test the env with zero actions.

utils

helpers.py

Some auxiliary functions are defined in this file.

- method `class_to_dict(obj)`:
Serialize an object to a dictionary by iterating through all its attributes and converting them to a dictionary.
- method `update_class_from_dict(obj, dict)`:
Set the attributes of an object from a dict by iterating through all the key-value pairs of the dict recursively.
- method `set_seed(seed)`:
To ensure reproducibility, set the seed for numpy, torch, and random.
- method `parse_sim_params(args, cfg)`:
Instantiate a `isaacgym.gymapi.SimParams` object and update it from the parsed arguments and the passed configuration.
- method `get_load_path(root, load_run=-1, checkpoint=-1)`:
Get the path to load the model from.
 - If `load_run` is specified, the load path is `root/{load_run}`, otherwise it is the latest run in `root`.
 - If `checkpoint` is specified, the model path is `model_{checkpoint}.pt`, otherwise it is the latest checkpoint in load path.
- method `update_cfg_from_args(env_cfg, cfg_train, args)`:
Update the environment and training configurations from the parsed arguments.
- method `get_args()`:
Parse the command line arguments using `isaacgym.gymutils.parse_arguments()`.
- method `export_policy_as_jit(actor_critic, path)`:
If the actor-critic model has an attribute "memory_a", use `PolicyExporterLSTM` to export the model as a TorchScript module. Otherwise, directly use `torch.jit.script()` to export the actor model.

class PolicyExporterLSTM(torch.nn.Module):

This class encapsulates a actor-critic model, extracting its actor network and LSTM-based memory, then exports the model as a TorchScript module.

- Constructor:
 - duplicate the actor network and LSTM memory from the actor-critic model.
 - register `hidden_state` and `cell_state` as persistent buffers.
- method `forward(self, x)`:
 1. process input data through the LSTM.
 2. update the hidden and cell states.
 3. pass the processed data through the actor network.
- method `export(self, path)`:
Move model to CPU and export it as a TorchScript module.

logger.py

Class `Logger` is designed to log and manage state and reward for RL tasks.

- Constructor:
Store some member variables:
 - `self.state_log`: state data
 - `self.rew_log`: reward data
 - `self.dt`: time step value used for plot function
 - `self.num_episodes`: a counter to keep track of the number of rewards logged
 - `self.plot_process`: a placeholder for a `Process` to plot data in separate process.
- Three `log_` method used to log data and a `reset(self)` function
- method `plot_states(self)`:
Start a process of `_plot(self)`
- method `_plot(self)`:
Use matplotlib to plot a 3 by 3 figure. From left to right, from top to bottom in sequence is:
 1. joint targets and measured positions
 2. joint velocities
 3. base velocity x
 4. base velocity y
 5. base angular velocity yaw
 6. base velocity z
 7. contact forces
 8. torque/velocity curves
 9. torques
- method `print_rewards(self)`:
Print average rewards per second and total number of episodes.

math.py

This file contains some math-related functions.

- method `quat_apply_yaw(quat, vec)`:
Copy the quaternion and reshape it to ensure proper dimensionality. Then it zeroes out the x and y (roll and pitch) components of the quaternion. After normalizing the modified quaternion, it applies the ywa only rotation to the input vector.
- method `wrap_to_pi(angles)`:
Wrap angles to $[-\pi, \pi]$ range.
- method `torch_rand_sqrt_float(lower, upper, shape, device)`:
 - Generates random float number that are more likely to be close to the **boundries** of the range.

- First using `2*torch.rand(*shape, device=device) - 1` to generate a random tensor in `[-1, 1]` range. Then apply a square root transformation with sign preservation. The resulting values are then be normalized to the range `[0, 1]` using `(r + 1.) / 2`. Finally the values are scaled to the desired range `[lower, upper]`.

task_registry.py

Class `TaskRegistry` is used to register and retrieve task-related configurations and classes in a structured way.

- `VecEnv` and `OnPolicyRunner` are imported from the `rs1_rl` package, which provides the RL env and runners.
- method `register(self, name, task_class, env_cfg, train_cfg)`:
register a new task by storing its class (in this project is `PointFoot`), environment configuration (`PointFootRoughCfg`) and training configuration (`PointFootRoughCfgPPO`).
- method `get_cfgs(self, name)`:
get the environment and training configurations of the task with the given name.
- method `make_env(self, name, args=None, env_cfg=None)`:
creates an environment either from a registered name (get configuration from the class member variables) or from the provided configuration file.
- method `make_alg_runner(self, env, name=None, args=None, train_cfg=None, log_root="default")`:
 - Creates the training algorithm either from a registered name (get configuration from the class member variables) or from the provided config file.
 - directory for logging is set to `log_root` if provided, otherwise to `default`.

terrain.py

This file contains a class `terrain` to generate terrain using `PointFootRoughCfg.terrain`.

- method `add_terrain_to_map(self, terrain, row, col)`
add a terrain segment to a larger map at the given row and column.
- method `make_terrain(self, choice, difficulty)`
 - first set params (slope, step_height, discrete_obstacles_height, gap_size...) of the terrain segment based on difficulty.
 - Based on "choice" and "proportions" in config, do a if-elif-else to generate the corresponding type of terrain.
- method `gap_terrain(terrain, gap_size, platform_size=1.)`:
generate a terrain with a platform and gap (very deep hole) around it.
- method `pit_terrain(terrain, depth, platform_size=1.)`:
generate a terrain with a pit in the middle.
- method `randomized_terrain(self)`:
randomly select choice and difficulty to generate a terrain. Repeat it.
- method `curriculum(self)`:
follow a curriculum to select choice and difficulty. Then generate a terrain and repeat the process.

- method `selected_terrain(self)`:
generate selected type of terrain based on `self.cfg.terrain_kwargs`.

pointfootMujoco

`rl_controller.py`

The `PointfootController` class implements a robotic controller for point-foot (PF), wheel-foot (WF) or sole-foot (SF) robot in Mujoco.

It need to be used along with the `SimulatorMujoco` class.

In this project it utilizes policy trained by Isaac Gym to control the PF robot's motion dynamically.

1. Initialization (`__init__` method):

- Load YAML configuration and ONNX policy model.
- Initialize the robot commands (`RoborCmd`), state (`RobotState`) and IMU data (`ImuData`) with default values.
- Set up callbacks to handle updates to robot state, IMU data, sensor joystick input, and diagnostics.

2. Configuration Loading (`load_config` method):

- Read configuration from YAML file.
- Assign configuration parameters to controller variables.
- Initialize variables for actions, observations, and commands.
- Initialize joint angles based on the initial configuration

3. Main Control Loop (`run` method):

- Iteratively calls the `update` method to execute the control loop.
- Ends safely by resetting robot command values

4. Modes of Operation

- `STAND` mode (`handle_stand_mode` method):
Smoothly moves the robot to a standing pose using linear interpolation of joint angles.
- `WALK` mode (`handle_walk_mode` method):
Uses the RL policy to generate actions for the robot.

5. Observations and Actions

- Observations (`compute_observation` method):
Final observations (with `np.clip`):
 - Base angular velocity (scaled)
 - Projected gravity vector
 - Joint positions (difference from initial angles, scaled)
 - Joint velocities (scaled)
 - Last actions applied to the robot
 - Scaled command inputs

- Actions (`compute_action` method):
Computes the actions based on the current observations using the policy session.

6. Callbacks

- `robot_state_callback` method:
Callback function to update the robot state from incoming data.
- `imu_data_callback` method:
Callback function to update IMU data from incoming data.
- `sensor_joy_callback` method:
Callback function for receiving sensor joy data
- `robot_diagnostic_callback` method:
Callback function for receiving diagnostic data

`simulator.py`

The `simulatorMujoco` class simulates and visualizes the robot in Mujoco.

1. **Load a robot model** from an XML file.
2. **Control the robot's joints** using commands received during the simulation.
3. **Collect data from sensors** (like IMU, joint positions, velocities, torque etc.)
4. **Visualize the simulation** using Mujoco viewer.

pointfoot-sdk-lowlevel

Python SDK for the low-level controller of the robot. Three architectures are available: `aarch64`, `amd64`, and `win`.

policy/PF_TRON1A

`policy/policy.onnx`

The trained policy model in ONNX format is stored here.

params.yaml

The parameters (`init_state`, `control`, `normalization`, etc.) used for simulation.

robot_description/PF_TRON1A

Meshes and URDF files for the robot model, which are used in the Mujoco.

robot-joystick

Store repo "limxdynamics/robot-joystick" for controlling the robot using a virtual joystick.

resources

This folder contains the URDF and mesh files for different robot models.

MISC

`install.sh`

A shell script to install the project dependencies.

`setup.py`

A script for setting up the `legged_gym` python package.